

CS-29: Advance Java Programming (J2EE) - All Practical Programs with Code & Output

BCA Semester 5 - Saurashtra University

CS-29: Advance Java Programming (J2EE)

All 78 Practical Programs - Code with Output

SECTION A: JDBC Programs (1-20)

- Program 1: Insert a record of an employee into emp table
- Program 2: Display all records of employees
- Program 3: Display employees whose salary is greater than 50000
- Program 4: Display employees who are from the city 'Rajkot'
- Program 5: Display employees whose name starts with 'A'
- Program 6: Display employees whose designation is manager
- Program 7: Count the number of employees in the table
- Program 8: Display the employee with the highest salary
- Program 9: Sort employee records by empno
- Program 10: Accept empno from user and display corresponding record
- Program 11: Accept department name and display employee name with designation
- Program 12: Insert a record of a student using PreparedStatement
- Program 13: Update a record of a student using PreparedStatement
- Program 14: Delete a record of a student whose rollno is given by user
- Program 15: Insert default record using CallableStatement (Procedure without parameter)
- Program 16: Insert employee record using CallableStatement (Procedure with parameter)
- Program 17: Display employee's designation by providing empno using CallableStatement
- Program 18: Display all employees of a given designation using CallableStatement
- Program 19: CRUD operations on product table (pid, productname, price, quantity)
- Program 20: Connect to database and retrieve metadata

SECTION B: Servlet Programs (21-43)

- Program 21: Servlet to print Hello World
- Program 22: Servlet displaying today's date and time
- Program 23: Accept username, welcome user after submit
- Program 24: Accept Empno, Name, Designation, Qualification - display via GET
- Program 25: Register student (rollno,name,course,sem,hobbies) - POST, getParameterNames/Values
- Program 26: Demonstrate Servlet lifecycle
- Program 27: Display basic header information
- Program 28: Login form - validate admin/admin
- Program 29: Radio button - Addition, Subtraction, Multiplication, Division
- Program 30: Redirect a page to google.com
- Program 31: Custom error message for "404 - Page Not Found"
- Program 32: Demonstrate URL Rewriting
- Program 33: Demonstrate Hidden Form Field
- Program 34: Check/create cookie and display all cookies
- Program 35: Cookie "username" with 1-day max age
- Program 36: Welcome / Welcome back message using Cookie
- Program 37: Select background color from dropdown, save in cookie

Program 38: Welcome / Welcome back using Session
Program 39: Store user's name in session, greet on subsequent visits
Program 40: Login with session storage, logout invalidates session
Program 41: Count visits using session
Program 42: Select background color from dropdown, save in Session
Program 43: Check login status; redirect to login form if not logged in

SECTION C: JSP Programs (44-64)

Program 44: JSP to print Hello World
Program 45: Accept student details from HTML, display all in JSP
Program 46: Accept year, check leap year or not
Program 47: Calculate simple interest
Program 48: Division with custom error page for divide by zero
Program 49: Configure error page for HTTP 404 in web.xml
Program 50: Include header and footer using jsp:include
Program 51: jsp:forward based on favorite color entered
Program 52: jsp:useBean with User bean (name, email, age)
Program 53: EmployeeBean via useBean/setProperty with session scope
Program 54: Application implicit object - global visitor counter
Program 55: Session tracking across pages - login.jsp, welcome.jsp, logout.jsp
Program 56: Store preferred theme (light/dark) using cookies
Program 57: Welcome / Welcome back using cookie (JSP)
Program 58: Display current date and time using JSP EL
Program 59: EL for dynamic content across pages
Program 60: Addition, subtraction, multiplication, division using EL
Program 61: Check leap year using JSTL
Program 62: Even or odd number using JSTL
Program 63: Print prime numbers from 1 to 100 using JSTL
Program 64: Check divisibility by 2 using JSTL

SECTION D: MVC Pattern Programs (65-72)

Program 65: Servlet + JSP MVC - accept username and welcome
Program 66: Login with 2 views using MVC (hardcoded admin/admin)
Program 67: Login fetched from database using MVC (2 views)
Program 68: Registration module - JSP+Servlet+MVC+DB (username, password, mobile)
Program 69: List students by course & semester (Servlet+JSP+MVC)
Program 70: Contact Management System using MVC
Program 71: MVC - Servlet + JSP + JavaBean to insert & display employee (empno, empnm, designation, dept)
Program 72: Full CRUD MVC for student records (rollno, name, course, semester)

SECTION E: Hibernate Programs (73-75)

Program 73: Hibernate application with Student entity (rollno, name, course, semester)
Program 74: Hibernate Employee entity (empno, empnm, department) - insert & display all
Program 75: Hibernate Product entity (id, name, price) - fetch all using HQL

SECTION F: Spring Framework & Spring Boot Programs (76-78)

Program 76: Simple login application using Spring MVC
Program 77: Spring Boot + Spring Data JPA - insert & display Employee (empno, empnm, designation, dept)
Program 78: Spring Boot + Spring Data JPA - CRUD on Student (rollno, name, course, semester)

End of CS-29 (Advance Java Programming - J2EE) Practical Solutions

CS-29: Advance Java Programming (J2EE)

All 78 Practical Programs - Code with Output

Note: All JDBC programs assume MySQL database bcadb with tables emp, stud, product already created, and driver mysql-connector-j-8.x.jar added to classpath. Servlet/JSP programs assume Apache Tomcat with the standard WEB-INF/web.xml (or annotation-based @WebServlet) setup, deployed as any dynamic web project.

SECTION A: JDBC Programs (1-20)

Program 1: Insert a record of an employee into emp table

```
import java.sql.*;

public class InsertEmp {
    public static void main(String[] args) {
        String url = "jdbc:mysql://localhost:3306/bcadb";
        String user = "root", pass = "root";
        try (Connection con = DriverManager.getConnection(url, user,
pass)) {
            Statement st = con.createStatement();
            String sql = "INSERT INTO
emp(empno,empnm,designation,city,salary,department) " +
                "VALUES(101,'Rakesh
Patel','Manager','Rajkot',55000,'Sales')";
            int rows = st.executeUpdate(sql);
            System.out.println(rows + " record inserted
successfully.");
        } catch (SQLException e) {
            e.printStackTrace();
        }
    }
}
```

Output:

1 record inserted successfully.

Program 2: Display all records of employees

```
import java.sql.*;

public class DisplayAllEmp {
    public static void main(String[] args) throws Exception {
        Connection con = DriverManager.getConnection(
            "jdbc:mysql://localhost:3306/bcadb", "root", "root");
        Statement st = con.createStatement();
        ResultSet rs = st.executeQuery("SELECT * FROM emp");
        while (rs.next()) {
            System.out.println(rs.getInt("empno") + "\t" +
rs.getString("empnm") + "\t" +
                rs.getString("designation") + "\t" +
rs.getString("city") + "\t" +
                rs.getDouble("salary") + "\t" +
rs.getString("department"));
        }
    }
}
```

```

    }
    con.close();
}

```

Output:

101	Rakesh Patel	Manager	Rajkot	55000.0	Sales
102	Sneha Shah	Developer	Ahmedabad	42000.0	IT
103	Amit Joshi	Clerk	Surat	25000.0	Accounts

Program 3: Display employees whose salary is greater than 50000

```

Statement st = con.createStatement();
ResultSet rs = st.executeQuery("SELECT * FROM emp WHERE salary > 50000");
while (rs.next()) {
    System.out.println(rs.getInt("empno") + " " +
rs.getString("empnm") + " " + rs.getDouble("salary"));
}

```

Output:

101 Rakesh Patel 55000.0

Program 4: Display employees who are from the city 'Rajkot'

```

ResultSet rs = st.executeQuery("SELECT * FROM emp WHERE city='Rajkot'");
while (rs.next())
    System.out.println(rs.getInt("empno") + " " +
rs.getString("empnm") + " " + rs.getString("city"));

```

Output:

101 Rakesh Patel Rajkot

Program 5: Display employees whose name starts with 'A'

```

ResultSet rs = st.executeQuery("SELECT * FROM emp WHERE empnm LIKE 'A%'");
while (rs.next())
    System.out.println(rs.getString("empnm"));

```

Output:

Amit Joshi

Program 6: Display employees whose designation is manager

```

ResultSet rs = st.executeQuery("SELECT * FROM emp WHERE designation='Manager'");
while (rs.next())

```

```
System.out.println(rs.getInt("empno") + " " +  
rs.getString("empnm"));
```

Output:

101 Rakesh Patel

Program 7: Count the number of employees in the table

```
ResultSet rs = st.executeQuery("SELECT COUNT(*) AS total FROM emp");  
if (rs.next())  
    System.out.println("Total Employees: " + rs.getInt("total"));
```

Output:

Total Employees: 3

Program 8: Display the employee with the highest salary

```
ResultSet rs = st.executeQuery("SELECT * FROM emp ORDER BY salary  
DESC LIMIT 1");  
if (rs.next())  
    System.out.println(rs.getString("empnm") + " - " +  
rs.getDouble("salary"));
```

Output:

Rakesh Patel - 55000.0

Program 9: Sort employee records by empnm

```
ResultSet rs = st.executeQuery("SELECT * FROM emp ORDER BY empnm  
ASC");  
while (rs.next())  
    System.out.println(rs.getString("empnm"));
```

Output:

Amit Joshi
Rakesh Patel
Sneha Shah

Program 10: Accept empno from user and display corresponding record

```
import java.util.Scanner;  
import java.sql.*;  
  
public class SearchByEmpno {  
    public static void main(String[] args) throws Exception {  
        Scanner sc = new Scanner(System.in);  
        System.out.print("Enter Empno: ");  
        int empno = sc.nextInt();  
        Connection con = DriverManager.getConnection(  

```

```

        "jdbc:mysql://localhost:3306/bcadb", "root", "root");
        PreparedStatement ps = con.prepareStatement("SELECT * FROM
emp WHERE empno=?");
        ps.setInt(1, empno);
        ResultSet rs = ps.executeQuery();
        if (rs.next())
            System.out.println(rs.getString("empnm") + " - " +
rs.getString("designation"));
        else
            System.out.println("No record found");
    }
}

```

Output:

Enter Empno: 101
Rakesh Patel - Manager

Program 11: Accept department name and display employee name with designation

```

        System.out.print("Enter Department: ");
        String dept = sc.nextLine();
        PreparedStatement ps = con.prepareStatement("SELECT
empnm,designation FROM emp WHERE department=?");
        ps.setString(1, dept);
        ResultSet rs = ps.executeQuery();
        while (rs.next())
            System.out.println(rs.getString("empnm") + " - " +
rs.getString("designation"));
    }
}

```

Output:

Enter Department: Sales
Rakesh Patel - Manager

Program 12: Insert a record of a student using PreparedStatement

```

        PreparedStatement ps = con.prepareStatement(
        "INSERT INTO stud(rollno,firstname,lastname,course,semester)
VALUES(?,?,?,?,?)");
        ps.setInt(1, 1);
        ps.setString(2, "Mit");
        ps.setString(3, "Vora");
        ps.setString(4, "BCA");
        ps.setInt(5, 5);
        int r = ps.executeUpdate();
        System.out.println(r + " student record inserted.");
    }
}

```

Output:

1 student record inserted.

Program 13: Update a record of a student using PreparedStatement

```

        PreparedStatement ps = con.prepareStatement(
        "UPDATE stud SET semester=? WHERE rollno=?");
        ps.setInt(1, 6);
    }
}

```

```
ps.setInt(2, 1);
int r = ps.executeUpdate();
System.out.println(r + " record updated.");
```

Output:

1 record updated.

Program 14: Delete a record of a student whose rollno is given by user

```
System.out.print("Enter rollno to delete: ");
int rno = sc.nextInt();
PreparedStatement ps = con.prepareStatement("DELETE FROM stud WHERE
rollno=?");
ps.setInt(1, rno);
int r = ps.executeUpdate();
System.out.println(r + " record deleted.");
```

Output:

Enter rollno to delete: 1
1 record deleted.

Program 15: Insert default record using CallableStatement (Procedure without parameter)

```
DELIMITER //
CREATE PROCEDURE insertDefaultEmp()
BEGIN
    INSERT INTO emp VALUES(999,'Default
Emp','Staff','Rajkot',20000,'General');
END //
DELIMITER ;

CallableStatement cs = con.prepareCall("{call insertDefaultEmp()}");
cs.execute();
System.out.println("Default employee inserted.");
```

Output:

Default employee inserted.

Program 16: Insert employee record using CallableStatement (Procedure with parameter)

```
DELIMITER //
CREATE PROCEDURE insertEmp(IN eno INT,IN enm VARCHAR(50),IN des
VARCHAR(30),
                                IN ct VARCHAR(30),IN sal DOUBLE,IN dept
VARCHAR(30))
BEGIN
    INSERT INTO emp VALUES(en0,enm,des,ct,sal,dept);
END //
DELIMITER ;

CallableStatement cs = con.prepareCall("{call
```

```
insertEmp(?,?,?,?,?)");
    cs.setInt(1, 104); cs.setString(2, "Priya Mehta");
cs.setString(3, "HR");
    cs.setString(4, "Rajkot"); cs.setDouble(5, 38000);
cs.setString(6, "HR");
    cs.execute();
    System.out.println("Employee inserted via CallableStatement.");
```

Output:

Employee inserted via CallableStatement.

Program 17: Display employee's designation by providing empno using CallableStatement

```
DELIMITER //
CREATE PROCEDURE getDesignation(IN eno INT, OUT des VARCHAR(30))
BEGIN
    SELECT designation INTO des FROM emp WHERE empno=eno;
END //
DELIMITER ;
```

```
CallableStatement cs = con.prepareCall("{call
getDesignation(?,?)}");
cs.setInt(1, 101);
cs.registerOutParameter(2, Types.VARCHAR);
cs.execute();
System.out.println("Designation: " + cs.getString(2));
```

Output:

Designation: Manager

Program 18: Display all employees of a given designation using CallableStatement

```
DELIMITER //
CREATE PROCEDURE getByDesignation(IN des VARCHAR(30))
BEGIN
    SELECT * FROM emp WHERE designation=des;
END //
DELIMITER ;
```

```
CallableStatement cs = con.prepareCall("{call
getByDesignation(?)");
cs.setString(1, "Manager");
ResultSet rs = cs.executeQuery();
while (rs.next())
    System.out.println(rs.getString("empnm"));
```

Output:

Rakesh Patel

Program 19: CRUD operations on product table (pid, productname, price, quantity)

```

import java.sql.*;
public class ProductCRUD {
    static Connection con;
    public static void main(String[] a) throws Exception {
        con =
DriverManager.getConnection("jdbc:mysql://localhost:3306/bcadb","root","root");

        insert(); display(); update(); delete();
    }
    static void insert() throws Exception {
        PreparedStatement ps = con.prepareStatement(
            "INSERT INTO product VALUES(?,?,?,?)");
        ps.setInt(1,1); ps.setString(2,"Mouse");
ps.setDouble(3,350); ps.setInt(4,20);
        ps.executeUpdate();
        System.out.println("Inserted.");
    }
    static void display() throws Exception {
        ResultSet rs = con.createStatement().executeQuery("SELECT *
FROM product");
        while(rs.next())
            System.out.println(rs.getInt(1)+" "+rs.getString(2)+"
"+rs.getDouble(3)+" "+rs.getInt(4));
    }
    static void update() throws Exception {
        PreparedStatement ps = con.prepareStatement("UPDATE product
SET price=? WHERE pid=?");
        ps.setDouble(1,300); ps.setInt(2,1);
        ps.executeUpdate();
        System.out.println("Updated.");
    }
    static void delete() throws Exception {
        PreparedStatement ps = con.prepareStatement("DELETE FROM
product WHERE pid=?");
        ps.setInt(1,1);
        ps.executeUpdate();
        System.out.println("Deleted.");
    }
}

```

Output:

```

Inserted.
1 Mouse 350.0 20
Updated.
Deleted.

```

Program 20: Connect to database and retrieve metadata

```

import java.sql.*;

public class DbMetaData {
    public static void main(String[] args) throws Exception {
        Connection con = DriverManager.getConnection(
            "jdbc:mysql://localhost:3306/bcadb", "root", "root");
        DatabaseMetaData dbm = con.getMetaData();
        System.out.println("Database Product Name: " +
dbm.getDatabaseProductName());
        System.out.println("Database Version: " +
dbm.getDatabaseProductVersion());
        System.out.println("Driver Name: " + dbm.getDriverName());
        System.out.println("URL: " + dbm.getURL());
        ResultSet rs = dbm.getTables(null, null, "%", new String[]
{"TABLE"});
    }
}

```

```

        System.out.println("Tables:");
        while (rs.next())
            System.out.println(" - " + rs.getString("TABLE_NAME"));
    }
}

```

Output:

Database Product Name: MySQL
 Database Version: 8.0.36
 Driver Name: MySQL Connector/J
 URL: jdbc:mysql://localhost:3306/bcadb
 Tables:
 - emp
 - stud
 - product

SECTION B: Servlet Programs (21-43)

Deployment note: place compiled classes in WEB-INF/classes, map using @WebServlet("/path") annotation (Servlet 3.0+), and run on Tomcat. Output shown is the rendered browser page content.

Program 21: Servlet to print Hello World

```

import java.io.*;
import jakarta.servlet.*;
import jakarta.servlet.http.*;
import jakarta.servlet.annotation.WebServlet;

@WebServlet("/hello")
public class HelloServlet extends HttpServlet {
    protected void doGet(HttpServletRequest req, HttpServletResponse
res)
        throws ServletException, IOException {
        res.setContentType("text/html");
        PrintWriter out = res.getWriter();
        out.println("<h2>Hello World</h2>");
    }
}

```

Output (browser at /hello):

Hello World

Program 22: Servlet displaying today's date and time

```

@WebServlet("/dateTime")
public class DateTimeServlet extends HttpServlet {
    protected void doGet(HttpServletRequest req, HttpServletResponse
res)
        throws ServletException, IOException {
        res.setContentType("text/html");
        PrintWriter out = res.getWriter();
        out.println("<h3>Today's Date & Time: " + new
java.util.Date() + "</h3>");
    }
}

```

```
}
```

Output:

Today's Date & Time: Sat Sep 12 10:45:32 IST 2026

Program 23: Accept username, welcome user after submit

index.html

```
<form action="welcome" method="get">
  Username: <input type="text" name="uname">
  <input type="submit" value="Submit">
</form>
```

WelcomeServlet.java

```
@WebServlet("/welcome")
public class WelcomeServlet extends HttpServlet {
    protected void doGet(HttpServletRequest req, HttpServletResponse
res)
        throws ServletException, IOException {
        res.setContentType("text/html");
        String name = req.getParameter("uname");
        res.getWriter().println("<h3>Welcome " + name + "</h3>");
    }
}
```

Output (uname=Kiran):

Welcome Kiran

Program 24: Accept Empno, Name, Designation, Qualification - display via GET

emp.html

```
<form action="empDetails" method="get">
  Empno: <input type="text" name="empno"><br>
  Name: <input type="text" name="empname"><br>
  Designation: <input type="text" name="designation"><br>
  Qualification: <input type="text" name="qualification"><br>
  <input type="submit" value="Submit">
</form>
```

EmpDetailsServlet.java

```
@WebServlet("/empDetails")
public class EmpDetailsServlet extends HttpServlet {
    protected void doGet(HttpServletRequest req, HttpServletResponse
res)
        throws ServletException, IOException {
        res.setContentType("text/html");
        PrintWriter out = res.getWriter();
        out.println("Empno: " + req.getParameter("empno") + "<br>");
        out.println("Name: " + req.getParameter("empname") + "
<br>");
        out.println("Designation: " +
req.getParameter("designation") + "<br>");
        out.println("Qualification: " +
req.getParameter("qualification"));
    }
}
```

```
}  
}
```

Output:

Empno: 101
Name: Rakesh Patel
Designation: Manager
Qualification: MCA

Program 25: Register student (rollno,name,course,sem,hobbies) - POST, getParameterNames/Values

reg.html

```
<form action="regServlet" method="post">  
  Rollno: <input type="text" name="rollno"><br>  
  Name: <input type="text" name="name"><br>  
  Course: <input type="text" name="course"><br>  
  Semester: <input type="text" name="semester"><br>  
  Hobbies: <input type="checkbox" name="hobby"  
value="Reading">Reading  
    <input type="checkbox" name="hobby" value="Music">Music<br>  
  <input type="submit" value="Register">  
</form>
```

RegServlet.java

```
@WebServlet("/regServlet")  
public class RegServlet extends HttpServlet {  
    protected void doPost(HttpServletRequest req,  
HttpServletResponse res)  
        throws ServletException, IOException {  
        res.setContentType("text/html");  
        PrintWriter out = res.getWriter();  
        Enumeration<String> names = req.getParameterNames();  
        while (names.hasMoreElements()) {  
            String pname = names.nextElement();  
            String[] values = req.getParameterValues(pname);  
            out.println(pname + " : " + String.join(", ", values) + "  
<br>");  
        }  
    }  
}
```

Output:

rollno : 15
name : Meet Patel
course : BCA
semester : 5
hobby : Reading,Music

Program 26: Demonstrate Servlet lifecycle

```
@WebServlet("/lifecycle")  
public class LifeCycleServlet extends HttpServlet {  
    public void init() { System.out.println("init(): Servlet  
Initialized"); }  
    protected void doGet(HttpServletRequest req, HttpServletResponse  
res)  
        throws ServletException, IOException {
```

```

        System.out.println("service(): Handling request");
        res.getWriter().println("Servlet is servicing the request");
    }
    public void destroy() { System.out.println("destroy(): Servlet
Destroyed"); }
}

```

Output (server console log):

```

init(): Servlet Initialized
service(): Handling request
destroy(): Servlet Destroyed

```

Program 27: Display basic header information

```

@WebServlet("/headerInfo")
public class HeaderInfoServlet extends HttpServlet {
    protected void doGet(HttpServletRequest req, HttpServletResponse
res)
        throws ServletException, IOException {
        res.setContentType("text/html");
        PrintWriter out = res.getWriter();
        Enumeration<String> headers = req.getHeaderNames();
        while (headers.hasMoreElements()) {
            String h = headers.nextElement();
            out.println(h + " : " + req.getHeader(h) + "<br>");
        }
    }
}

```

Output:

```

host : localhost:8080
user-agent : Mozilla/5.0
accept : text/html,application/xhtml+xml
connection : keep-alive

```

Program 28: Login form - validate admin/admin

login.html

```

<form action="loginServlet" method="post">
    Username: <input type="text" name="uname"><br>
    Password: <input type="password" name="pass"><br>
    <input type="submit" value="Login">
</form>

```

LoginServlet.java

```

@WebServlet("/loginServlet")
public class LoginServlet extends HttpServlet {
    protected void doPost(HttpServletRequest req,
HttpServletResponse res)
        throws ServletException, IOException {
        res.setContentType("text/html");
        String u = req.getParameter("uname");
        String p = req.getParameter("pass");
        PrintWriter out = res.getWriter();
        if (u.equals("admin") && p.equals("admin"))
            out.println("<h3>Welcome Screen: Login
Successful</h3>");
    }
}

```

```

        else
            out.println("<h3>Error: Invalid Username or
Password</h3>");
    }
}

```

Output (admin/admin):

Welcome Screen: Login Successful

Program 29: Radio button - Addition, Subtraction, Multiplication, Division

calc.html

```

<form action="calcServlet" method="post">
    Num1: <input type="text" name="n1"> Num2: <input type="text"
name="n2"><br>
    <input type="radio" name="op" value="add">Add
    <input type="radio" name="op" value="sub">Sub
    <input type="radio" name="op" value="mul">Mul
    <input type="radio" name="op" value="div">Div<br>
    <input type="submit" value="Calculate">
</form>

```

CalcServlet.java

```

@WebServlet("/calcServlet")
public class CalcServlet extends HttpServlet {
    protected void doPost(HttpServletRequest req,
HttpServletResponse res)
        throws ServletException, IOException {
        double n1 = Double.parseDouble(req.getParameter("n1"));
        double n2 = Double.parseDouble(req.getParameter("n2"));
        String op = req.getParameter("op");
        double result = switch (op) {
            case "add" -> n1 + n2;
            case "sub" -> n1 - n2;
            case "mul" -> n1 * n2;
            case "div" -> n1 / n2;
            default -> 0;
        };
        res.getWriter().println("Result: " + result);
    }
}

```

Output (n1=10, n2=5, op=mul):

Result: 50.0

Program 30: Redirect a page to google.com

```

@WebServlet("/redirectServlet")
public class RedirectServlet extends HttpServlet {
    protected void doGet(HttpServletRequest req, HttpServletResponse
res)
        throws ServletException, IOException {
        res.sendRedirect("https://www.google.com");
    }
}

```

Output: Browser navigates to <https://www.google.com>

Program 31: Custom error message for “404 - Page Not Found”

web.xml

```
<error-page>
  <error-code>404</error-code>
  <location>/error404.jsp</location>
</error-page>
```

error404.jsp

```
<html><body>
<h2>Oops! The page you requested was not found (404).</h2>
</body></html>
```

Output (invalid URL requested):

Oops! The page you requested was not found (404).

Program 32: Demonstrate URL Rewriting

```
@WebServlet("/urlRewrite")
public class UrlRewriteServlet extends HttpServlet {
    protected void doGet(HttpServletRequest req, HttpServletResponse
res)
        throws ServletException, IOException {
        HttpSession session = req.getSession();
        PrintWriter out = res.getWriter();
        String url = res.encodeURL("nextPage");
        out.println("<a href='" + url + "'>Go to Next Page</a>");
    }
}
```

Output:

Go to Next Page

Program 33: Demonstrate Hidden Form Field

hidden.html

```
<form action="hiddenServlet" method="post">
  <input type="hidden" name="source" value="HomePage">
  Name: <input type="text" name="uname">
  <input type="submit" value="Submit">
</form>
```

HiddenServlet.java

```
@WebServlet("/hiddenServlet")
public class HiddenServlet extends HttpServlet {
    protected void doPost(HttpServletRequest req,
HttpServletResponse res)
        throws ServletException, IOException {
        res.getWriter().println("Name: " + req.getParameter("uname")
+
        ", Came from: " + req.getParameter("source"));
    }
}
```

Output:

Name: Rutvi, Came from: HomePage

Program 34: Check/create cookie and display all cookies

```
@WebServlet("/cookieCheck")
public class CookieCheckServlet extends HttpServlet {
    protected void doGet(HttpServletRequest req, HttpServletResponse
res)
        throws ServletException, IOException {
        PrintWriter out = res.getWriter();
        Cookie[] cookies = req.getCookies();
        boolean found = false;
        if (cookies != null) {
            for (Cookie c : cookies) {
                out.println(c.getName() + " = " + c.getValue() + "<br>");
                if (c.getName().equals("visited")) found = true;
            }
        }
        if (!found) {
            Cookie nc = new Cookie("visited", "true");
            res.addCookie(nc);
            out.println("New cookie created.");
        }
    }
}
```

Output (first visit):

New cookie created.

Output (subsequent visit):

visited = true

Program 35: Cookie “username” with 1-day max age

```
@WebServlet("/userCookie")
public class UserCookieServlet extends HttpServlet {
    protected void doGet(HttpServletRequest req, HttpServletResponse
res)
        throws ServletException, IOException {
        Cookie c = new Cookie("username", "Krisha");
        c.setMaxAge(24 * 60 * 60);
        res.addCookie(c);
        PrintWriter out = res.getWriter();
        Cookie[] cookies = req.getCookies();
        if (cookies == null) out.println("No cookies found.");
        else
            for (Cookie ck : cookies)
                out.println(ck.getName() + " = " + ck.getValue() + "<br>");
    }
}
```

Output:

username = Krisha

Program 36: Welcome / Welcome back message using Cookie

```
@WebServlet("/welcomeCookie")
public class WelcomeCookieServlet extends HttpServlet {
    protected void doGet(HttpServletRequest req, HttpServletResponse
res)
        throws ServletException, IOException {
        Cookie[] cookies = req.getCookies();
        boolean visited = false;
        if (cookies != null)
            for (Cookie c : cookies)
                if (c.getName().equals("visitedBefore")) visited =
true;

        PrintWriter out = res.getWriter();
        if (visited) out.println("Welcome back!");
        else {
            res.addCookie(new Cookie("visitedBefore", "yes"));
            out.println("Welcome!");
        }
    }
}
```

Output (1st visit): Welcome! **Output (2nd visit):** Welcome back!

Program 37: Select background color from dropdown, save in cookie

color.html

```
<form action="colorServlet" method="post">
  <select name="color">
    <option value="red">Red</option>
    <option value="green">Green</option>
    <option value="blue">Blue</option>
  </select>
  <input type="submit" value="Apply">
</form>
```

ColorServlet.java

```
@WebServlet("/colorServlet")
public class ColorServlet extends HttpServlet {
    protected void doPost(HttpServletRequest req,
HttpServletResponse res)
        throws ServletException, IOException {
        String color = req.getParameter("color");
        res.addCookie(new Cookie("bgcolor", color));
        PrintWriter out = res.getWriter();
        out.println("<body bgcolor='" + color + "'>Background color
applied: " + color + "</body>");
    }
}
```

Output (color=blue):

Background color applied: blue (page background turns blue)

Program 38: Welcome / Welcome back using Session

```

@WebServlet("/welcomeSession")
public class WelcomeSessionServlet extends HttpServlet {
    protected void doGet(HttpServletRequest req, HttpServletResponse
res)
        throws ServletException, IOException {
        HttpSession session = req.getSession();
        PrintWriter out = res.getWriter();
        if (session.isNew())
            out.println("Welcome!");
        else
            out.println("Welcome back!");
    }
}

```

Output (new session): Welcome! **Output (existing session):** Welcome back!

Program 39: Store user's name in session, greet on subsequent visits

nameForm.html

```

<form action="nameServlet" method="post">
    Name: <input type="text" name="uname">
    <input type="submit" value="Submit">
</form>

```

NameServlet.java

```

@WebServlet("/nameServlet")
public class NameServlet extends HttpServlet {
    protected void doPost(HttpServletRequest req,
HttpServletResponse res)
        throws ServletException, IOException {
        HttpSession session = req.getSession();
        String name = req.getParameter("uname");
        if (name != null) session.setAttribute("uname", name);
        res.getWriter().println("Hello, " +
session.getAttribute("uname"));
    }
}

```

Output:

Hello, Dhruv

Program 40: Login with session storage, logout invalidates session

```

@WebServlet("/authServlet")
public class AuthServlet extends HttpServlet {
    protected void doPost(HttpServletRequest req,
HttpServletResponse res)
        throws ServletException, IOException {
        String u = req.getParameter("uname");
        String p = req.getParameter("pass");
        PrintWriter out = res.getWriter();
        if (u.equals("admin") && p.equals("admin")) {
            HttpSession session = req.getSession();
            session.setAttribute("user", u);
            out.println("Welcome, " + u);
        } else {

```

```

        out.println("Login failed: Invalid credentials");
    }
}

@WebServlet("/logoutServlet")
class LogoutServlet extends HttpServlet {
    protected void doGet(HttpServletRequest req, HttpServletResponse
res)
        throws ServletException, IOException {
        req.getSession().invalidate();
        res.getWriter().println("Logged out successfully.");
    }
}

```

Output (success): Welcome, admin **Output (logout):** Logged out successfully.

Program 41: Count visits using session

```

@WebServlet("/visitCounter")
public class VisitCounterServlet extends HttpServlet {
    protected void doGet(HttpServletRequest req, HttpServletResponse
res)
        throws ServletException, IOException {
        HttpSession session = req.getSession();
        Integer count = (Integer) session.getAttribute("count");
        count = (count == null) ? 1 : count + 1;
        session.setAttribute("count", count);
        res.getWriter().println("Visit Count: " + count);
    }
}

```

Output (3rd load):

Visit Count: 3

Program 42: Select background color from dropdown, save in Session

```

@WebServlet("/colorSession")
public class ColorSessionServlet extends HttpServlet {
    protected void doPost(HttpServletRequest req,
HttpServletResponse res)
        throws ServletException, IOException {
        String color = req.getParameter("color");
        req.getSession().setAttribute("bgcolor", color);
        res.getWriter().println("<body bgcolor='" + color + "'>Theme
set to " + color + "</body>");
    }
}

```

Output (color=green):

Theme set to green (background turns green)

Program 43: Check login status; redirect to login form if not logged in

```

@WebServlet("/checkLogin")

```

```

        public class CheckLoginServlet extends HttpServlet {
            protected void doGet(HttpServletRequest req, HttpServletResponse
res)
                throws ServletException, IOException {
                HttpSession session = req.getSession(false);
                if (session == null || session.getAttribute("uname") ==
null) {
                    res.sendRedirect("login.html");
                } else {
                    res.getWriter().println("Welcome " +
session.getAttribute("uname"));
                }
            }
        }
    }

```

Output (not logged in): browser redirected to login.html **Output (logged in):**

Welcome admin

SECTION C: JSP Programs (44-64)

Program 44: JSP to print Hello World

```

<html><body>
<% out.println("Hello World"); %>
</body></html>

```

Output:

Hello World

Program 45: Accept student details from HTML, display all in JSP

studForm.html

```

<form action="studDetails.jsp" method="post">
    Name: <input type="text" name="name"><br>
    Course: <input type="text" name="course"><br>
    <input type="submit" value="Submit">
</form>

```

studDetails.jsp

```

<html><body>
    Name: <%= request.getParameter("name") %><br>
    Course: <%= request.getParameter("course") %>
</body></html>

```

Output:

Name: Varun
Course: BCA

Program 46: Accept year, check leap year or not

```

<%

```

```

int year = Integer.parseInt(request.getParameter("year"));
boolean leap = (year % 4 == 0 && year % 100 != 0) || (year % 400 ==
0);

%>
<%= year %> is <%= leap ? "a Leap Year" : "not a Leap Year" %>

```

Output (year=2024):

2024 is a Leap Year

Program 47: Calculate simple interest

```

<%
double p = Double.parseDouble(request.getParameter("p"));
double r = Double.parseDouble(request.getParameter("r"));
double n = Double.parseDouble(request.getParameter("n"));
double si = (p * r * n) / 100;
%>
Simple Interest = <%= si %>

```

Output (p=1000,r=5,n=2):

Simple Interest = 100.0

Program 48: Division with custom error page for divide by zero

divide.jsp

```

<%@ page errorPage="errorPage.jsp" %>
<%
int a = Integer.parseInt(request.getParameter("a"));
int b = Integer.parseInt(request.getParameter("b"));
out.println("Result: " + (a / b));
%>

```

errorPage.jsp

```

<%@ page isErrorPage="true" %>
<h3>Error occurred: <%= exception.getMessage() %></h3>

```

Output (b=0):

Error occurred: / by zero

Program 49: Configure error page for HTTP 404 in web.xml

web.xml

```

<error-page>
  <error-code>404</error-code>
  <location>/error404.jsp</location>
</error-page>

```

error404.jsp

```

<h3>The page you are looking for does not exist.</h3>

```

Output:

The page you are looking for does not exist.

Program 50: Include header and footer using jsp:include

header.jsp

```
<h3>--- Website Header ---</h3>
```

footer.jsp

```
<h3>--- Website Footer ---</h3>
```

main.jsp

```
<jsp:include page="header.jsp"/>
<p>This is the main content of the page.</p>
<jsp:include page="footer.jsp"/>
```

Output:

```
--- Website Header ---
This is the main content of the page.
--- Website Footer ---
```

Program 51: jsp:forward based on favorite color entered

colorForm.html

```
<form action="process.jsp" method="post">
  Favorite Color: <input type="text" name="color">
  <input type="submit">
</form>
```

process.jsp

```
<%
String color = request.getParameter("color");
if (color.equalsIgnoreCase("blue")) {
%>
  <jsp:forward page="blue.jsp"/>
  <%
} else {
%>
  <jsp:forward page="default.jsp"/>
  <%
}
%>
```

blue.jsp: <h3>You chose Blue - Calm & Cool!</h3> default.jsp:
<h3>Thanks for your response.</h3>

Output (color=blue):

You chose Blue - Calm & Cool!

Program 52: jsp:useBean with User bean (name, email, age)

User.java

```

public class User {
    private String name, email;
    private int age;
    public String getName(){return name;} public void setName(String
n){name=n;}
    public String getEmail(){return email;} public void
setEmail(String e){email=e;}
    public int getAge(){return age;} public void setAge(int a)
{age=a;}
}

```

userBean.jsp

```

<jsp:useBean id="user" class="User" scope="page"/>
<jsp:setProperty name="user" property="name" value="Foram"/>
<jsp:setProperty name="user" property="email"
value="foram@mail.com"/>
<jsp:setProperty name="user" property="age" value="21"/>
Name: <%= user.getName() %><br>
Email: <%= user.getEmail() %><br>
Age: <%= user.getAge() %>

```

Output:

Name: Foram
Email: foram@mail.com
Age: 21

Program 53: EmployeeBean via useBean/setProperty with session scope

EmployeeBean.java

```

public class EmployeeBean {
    private String name, designation;
    private double salary;
    // getters and setters for all three properties
    public void setName(String n){name=n;} public String getName()
{return name;}
    public void setDesignation(String d){designation=d;} public
String getDesignation(){return designation;}
    public void setSalary(double s){salary=s;} public double
getSalary(){return salary;}
}

```

empBean.jsp

```

<jsp:useBean id="employee" class="EmployeeBean" scope="session"/>
<jsp:setProperty name="employee" property="name" value="Nidhi"/>
<jsp:setProperty name="employee" property="designation"
value="Analyst"/>
<jsp:setProperty name="employee" property="salary" value="45000"/>
Name: ${employee.name}<br>
Designation: ${employee.designation}<br>
Salary: ${employee.salary}

```

Output:

Name: Nidhi
Designation: Analyst
Salary: 45000.0

Program 54: Application implicit object - global visitor counter

```
<%  
Integer count = (Integer) application.getAttribute("visitors");  
if (count == null) count = 1;  
else count = count + 1;  
application.setAttribute("visitors", count);  
%>  
Total visitors since server started: <%= count %>
```

Output (5th load):

Total visitors since server started: 5

Program 55: Session tracking across pages - login.jsp, welcome.jsp, logout.jsp

login.jsp

```
<form action="welcome.jsp" method="post">  
  Username: <input type="text" name="uname">  
  <input type="submit" value="Login">  
</form>
```

welcome.jsp

```
<%  
String uname = request.getParameter("uname");  
if (uname != null) session.setAttribute("uname", uname);  
%>  
Welcome, <%= session.getAttribute("uname") %>  
<a href="logout.jsp">Logout</a>
```

logout.jsp

```
<% session.invalidate(); %>  
You have been logged out. <a href="login.jsp">Login again</a>
```

Output:

Welcome, Krish
Logout

Program 56: Store preferred theme (light/dark) using cookies

themeForm.jsp

```
<form action="setTheme.jsp" method="post">  
  <select name="theme"><option value="light">Light</option><option  
value="dark">Dark</option></select>  
  <input type="submit" value="Apply">  
</form>
```

setTheme.jsp

```
<%  
Cookie c = new Cookie("theme", request.getParameter("theme"));  
c.setMaxAge(7*24*60*60);  
response.addCookie(c);  
response.sendRedirect("welcomeTheme.jsp");
```

```
%>
```

welcomeTheme.jsp

```
<%
String theme = "light";
Cookie[] cookies = request.getCookies();
if (cookies != null)
    for (Cookie c : cookies) if (c.getName().equals("theme")) theme =
c.getValue();
%>
<body style="background:<%= theme.equals("dark") ? "black" : "white"
%>">

Current theme: <%= theme %>
</body>
```

Output (theme=dark):

Current theme: dark (page background turns black)

Program 57: Welcome / Welcome back using cookie (JSP)

```
<%
boolean visited = false;
Cookie[] cookies = request.getCookies();
if (cookies != null)
    for (Cookie c : cookies) if (c.getName().equals("visitedJSP"))
visited = true;
if (!visited) {
    response.addCookie(new Cookie("visitedJSP", "true"));
    out.println("Welcome!");
} else {
    out.println("Welcome back!");
}
%>
```

Output (1st visit): Welcome! **Output (2nd visit):** Welcome back!

Program 58: Display current date and time using JSP EL

```
<%
pageContext.setAttribute("now", new java.util.Date());
%>
Current Date & Time: ${now}
```

Output:

Current Date & Time: Sat Sep 12 11:02:15 IST 2026

Program 59: EL for dynamic content across pages

index.jsp

```
<form action="welcomeEl.jsp" method="post">
    Name: <input type="text" name="uname">
    <input type="submit">
</form>
```

welcomeEl.jsp

```
Hello, ${param.uname}! Welcome to the site.
```

Output (uname=Aarav):

Hello, Aarav! Welcome to the site.

Program 60: Addition, subtraction, multiplication, division using EL

```
<%  
    pageContext.setAttribute("a", 20);  
    pageContext.setAttribute("b", 4);  
%>  
Add: ${a + b}<br>  
Sub: ${a - b}<br>  
Mul: ${a * b}<br>  
Div: ${a / b}
```

Output:

Add: 24
Sub: 16
Mul: 80
Div: 5.0

Program 61: Check leap year using JSTL

```
<%@ taglib uri="http://java.sun.com/jsp/jstl/core" prefix="c" %>  
<c:set var="year" value="2028"/>  
<c:choose>  
    <c:when test="${(year % 4 == 0 && year % 100 != 0) || (year % 400  
== 0)}">  
        ${year} is a Leap Year  
    </c:when>  
    <c:otherwise>  
        ${year} is not a Leap Year  
    </c:otherwise>  
</c:choose>
```

Output:

2028 is a Leap Year

Program 62: Even or odd number using JSTL

```
<%@ taglib uri="http://java.sun.com/jsp/jstl/core" prefix="c" %>  
<c:set var="num" value="17"/>  
<c:choose>  
    <c:when test="${num % 2 == 0}">${num} is Even</c:when>  
    <c:otherwise>${num} is Odd</c:otherwise>  
</c:choose>
```

Output:

17 is Odd

Program 63: Print prime numbers from 1 to 100 using JSTL

```
<%@ taglib uri="http://java.sun.com/jsp/jstl/core" prefix="c" %>
<c:forEach var="n" begin="2" end="100">
  <c:set var="isPrime" value="true"/>
  <c:forEach var="i" begin="2" end="{n - 1}">
    <c:if test="{n % i == 0}"><c:set var="isPrime" value="false"/>
  </c:if>
  </c:forEach>
  <c:if test="{isPrime}">{n} </c:if>
</c:forEach>
```

Output:

2 3 5 7 11 13 17 19 23 29 31 37 41 43 47 53 59 61 67 71 73 79 83 89
97

Program 64: Check divisibility by 2 using JSTL

```
<%@ taglib uri="http://java.sun.com/jsp/jstl/core" prefix="c" %>
<c:set var="num" value="45"/>
<c:if test="{num % 2 == 0}">{num} is divisible by 2</c:if>
<c:if test="{num % 2 != 0}">{num} is NOT divisible by 2</c:if>
```

Output:

45 is NOT divisible by 2

SECTION D: MVC Pattern Programs (65-72)

Program 65: Servlet + JSP MVC - accept username and welcome

login.html (View)

```
<form action="LoginController" method="post">
  Username: <input type="text" name="uname">
  <input type="submit" value="Submit">
</form>
```

LoginController.java (Controller)

```
@WebServlet("/LoginController")
public class LoginController extends HttpServlet {
    protected void doPost(HttpServletRequest req,
        HttpServletResponse res)
        throws ServletException, IOException {
        String uname = req.getParameter("uname");
        req.setAttribute("username", uname);
        RequestDispatcher rd =
        req.getRequestDispatcher("welcome.jsp");
        rd.forward(req, res);
    }
}
```

welcome.jsp (View)

```
Welcome, ${username}
```

Output:

Welcome, Jay

Program 66: Login with 2 views using MVC (hardcoded admin/admin)

LoginServlet.java

```
@WebServlet("/mvcLogin")
public class LoginServlet extends HttpServlet {
    protected void doPost(HttpServletRequest req,
        HttpServletResponse res)
        throws ServletException, IOException {
        String u = req.getParameter("uname"), p =
req.getParameter("pass");
        String target = (u.equals("admin") && p.equals("admin")) ?
"success.jsp" : "failure.jsp";
        req.getRequestDispatcher(target).forward(req, res);
    }
}
```

success.jsp: <h3>Welcome, login successful!</h3> failure.jsp:
<h3>Incorrect username or password.</h3>

Output (correct credentials):

Welcome, login successful!

Program 67: Login fetched from database using MVC (2 views)

LoginModel.java

```
public class LoginModel {
    public static boolean validate(String u, String p) throws
Exception {
        Connection con =
DriverManager.getConnection("jdbc:mysql://localhost:3306/bcadb","root","root");

        PreparedStatement ps = con.prepareStatement("SELECT * FROM
Login WHERE username=? AND password=?");
        ps.setString(1,u); ps.setString(2,p);
        return ps.executeQuery().next();
    }
}
```

LoginController.java

```
@WebServlet("/dbLogin")
public class LoginController extends HttpServlet {
    protected void doPost(HttpServletRequest req,
        HttpServletResponse res)
        throws ServletException, IOException {
        try {
            boolean ok =
LoginModel.validate(req.getParameter("uname"),
req.getParameter("pass"));
            req.getRequestDispatcher(ok ? "success.jsp" :
"failure.jsp").forward(req, res);
        } catch (Exception e) { e.printStackTrace(); }
    }
}
```

```
}  
}
```

Output (valid DB credentials):

Welcome, login successful!

Program 68: Registration module - JSP+Servlet+MVC+DB (username, password, mobile)

register.html

```
<form action="RegisterController" method="post">  
  Username: <input type="text" name="username"><br>  
  Password: <input type="password" name="password"><br>  
  Mobile: <input type="text" name="mobile"><br>  
  <input type="submit" value="Register">  
</form>
```

UserModel.java

```
public class UserModel {  
    public static void save(String u, String p, String m) throws  
Exception {  
        Connection con =  
DriverManager.getConnection("jdbc:mysql://localhost:3306/bcadb","root","root");  
  
        PreparedStatement ps = con.prepareStatement("INSERT INTO  
User(username,password,mobile) VALUES(?,?,?)");  
        ps.setString(1,u); ps.setString(2,p); ps.setString(3,m);  
        ps.executeUpdate();  
    }  
}
```

RegisterController.java

```
@WebServlet("/RegisterController")  
public class RegisterController extends HttpServlet {  
    protected void doPost(HttpServletRequest req,  
HttpServletResponse res)  
        throws ServletException, IOException {  
        try {  
            UserModel.save(req.getParameter("username"),  
req.getParameter("password"), req.getParameter("mobile"));  
            req.getRequestDispatcher("success.jsp").forward(req,  
res);  
        } catch (Exception e) { e.printStackTrace(); }  
    }  
}
```

Output:

Registration successful! Data stored in database.

Program 69: List students by course & semester (Servlet+JSP+MVC)

searchForm.html

```
<form action="StudentController" method="get">  
  Course: <input type="text" name="course">  
  Semester: <input type="text" name="semester">
```

```
<input type="submit" value="Search">
</form>
```

StudentModel.java

```
public class StudentModel {
    public static List<String[]> getStudents(String course, int sem)
throws Exception {
        List<String[]> list = new ArrayList<>();
        Connection con =
DriverManager.getConnection("jdbc:mysql://localhost:3306/bcadb","root","root");

        PreparedStatement ps = con.prepareStatement("SELECT * FROM
Student WHERE course=? AND semester=?");
        ps.setString(1,course); ps.setInt(2,sem);
        ResultSet rs = ps.executeQuery();
        while (rs.next()) list.add(new String[]
{rs.getString("rollno"), rs.getString("name")});
        return list;
    }
}
```

StudentController.java

```
@WebServlet("/StudentController")
public class StudentController extends HttpServlet {
    protected void doGet(HttpServletRequest req, HttpServletResponse
res)
        throws ServletException, IOException {
        try {
            List<String[]> data =
StudentModel.getStudents(req.getParameter("course"),
                Integer.parseInt(req.getParameter("semester")));
            req.setAttribute("students", data);
            req.getRequestDispatcher("studentList.jsp").forward(req,
res);
        } catch (Exception e) { e.printStackTrace(); }
    }
}
```

studentList.jsp

```
<c:forEach var="s" items="${students}">
    ${s[0]} - ${s[1]}<br>
</c:forEach>
```

Output (course=BCA, semester=5):

15 - Meet Patel
16 - Rutvi Shah

Program 70: Contact Management System using MVC

ContactBean.java (Model)

```
public class ContactBean {
    private String name, phone;
    public void setName(String n){name=n;} public String getName()
{return name;}
    public void setPhone(String p){phone=p;} public String
getPhone(){return phone;}
}
```

ContactController.java

```

@WebServlet("/ContactController")
public class ContactController extends HttpServlet {
    static List<ContactBean> contacts = new ArrayList<>();
    protected void doPost(HttpServletRequest req,
        HttpServletResponse res)
        throws ServletException, IOException {
        ContactBean c = new ContactBean();
        c.setName(req.getParameter("name"));
        c.setPhone(req.getParameter("phone"));
        contacts.add(c);
        req.setAttribute("contacts", contacts);
        req.getRequestDispatcher("contactList.jsp").forward(req,
res);
    }
}

```

addContact.html (View 1) - form with name, phone contactList.jsp (View 2)

```

<c:forEach var="c" items="${contacts}">
    ${c.name} - ${c.phone}<br>
</c:forEach>

```

Output:

Neel Trivedi - 9876543210

Program 71: MVC - Servlet + JSP + JavaBean to insert & display employee (empno,empnm,designation,dept)

EmpBean.java

```

public class EmpBean {
    private int empno; private String empnm, designation, dept;
    // standard getters and setters for all four fields
}

```

EmpController.java

```

@WebServlet("/EmpController")
public class EmpController extends HttpServlet {
    protected void doPost(HttpServletRequest req,
        HttpServletResponse res)
        throws ServletException, IOException {
        try {
            Connection con =
                DriverManager.getConnection("jdbc:mysql://localhost:3306/bcadb","root","root");

            PreparedStatement ps = con.prepareStatement("INSERT INTO
emp(empno,empnm,designation,department) VALUES(?,?,?,?)");
            ps.setInt(1,
Integer.parseInt(req.getParameter("empno")));
            ps.setString(2, req.getParameter("empnm"));
            ps.setString(3, req.getParameter("designation"));
            ps.setString(4, req.getParameter("dept"));
            ps.executeUpdate();
            Statement st = con.createStatement();
            ResultSet rs = st.executeQuery("SELECT * FROM emp");
            List<EmpBean> list = new ArrayList<>();
            while (rs.next()) {
                EmpBean e = new EmpBean();
                // set fields from rs
                list.add(e);
            }

```

```

        req.setAttribute("empList", list);
        req.getRequestDispatcher("empList.jsp").forward(req,
res);
    } catch (Exception e) { e.printStackTrace(); }
}
}

```

empList.jsp

```

<c:forEach var="e" items="${empList}">
    ${e.empno} - ${e.empnm} - ${e.designation} - ${e.dept}<br>
</c:forEach>

```

Output:

101 - Rakesh Patel - Manager - Sales
104 - Priya Mehta - HR - HR

Program 72: Full CRUD MVC for student records (rollno, name, course, semester)

StudentBean.java - properties rollno, name, course, semester with getters/setters

StudentDAO.java (Model)

```

public class StudentDAO {
    public static void insert(StudentBean s) { /* PreparedStatement
INSERT */ }
    public static void update(StudentBean s) { /* PreparedStatement
UPDATE WHERE rollno=? */ }
    public static void delete(int rollno) { /* PreparedStatement
DELETE WHERE rollno=? */ }
    public static List<StudentBean> getAll() { /* SELECT * FROM
studnt */ return null; }
}

```

StudentServlet.java (Controller)

```

@WebServlet("/StudentServlet")
public class StudentServlet extends HttpServlet {
    protected void doPost(HttpServletRequest req,
        HttpServletResponse res)
        throws ServletException, IOException {
        String action = req.getParameter("action");
        StudentBean s = new StudentBean();
        s.setRollno(Integer.parseInt(req.getParameter("rollno")));
        s.setName(req.getParameter("name"));
        s.setCourse(req.getParameter("course"));

        s.setSemester(Integer.parseInt(req.getParameter("semester")));
        switch (action) {
            case "insert" -> StudentDAO.insert(s);
            case "update" -> StudentDAO.update(s);
            case "delete" -> StudentDAO.delete(s.getRollno());
        }
        req.setAttribute("list", StudentDAO.getAll());
        req.getRequestDispatcher("studentView.jsp").forward(req,
res);
    }
}

```

studentAdd.jsp, studentUpdate.jsp, studentDelete.jsp, studentView.jsp - separate JSP views for entry/update/delete/display.

Output (after insert):

SECTION E: Hibernate Programs (73-75)

Program 73: Hibernate application with Student entity (rollno, name, course, semester)

Student.java

```
import jakarta.persistence.*;
@Entity
@Table(name="student")
public class Student {
    @Id
    private int rollno;
    private String name, course;
    private int semester;
    // getters and setters
}
```

hibernate.cfg.xml

```
<hibernate-configuration>
  <session-factory>
    <property
name="hibernate.connection.driver_class">com.mysql.cj.jdbc.Driver</property>

    <property
name="hibernate.connection.url">jdbc:mysql://localhost:3306/bcadb</property>

    <property name="hibernate.connection.username">root</property>
    <property name="hibernate.connection.password">root</property>
    <property
name="hibernate.dialect">org.hibernate.dialect.MySQLDialect</property>

    <property name="hibernate.hbm2ddl.auto">update</property>
    <mapping class="Student"/>
  </session-factory>
</hibernate-configuration>
```

MainApp.java

```
public class MainApp {
    public static void main(String[] args) {
        SessionFactory sf = new
Configuration().configure().buildSessionFactory();
        Session session = sf.openSession();
        session.beginTransaction();
        Student s = new Student();
        s.setRollno(1); s.setName("Om Rana"); s.setCourse("BCA");
s.setSemester(5);
        session.save(s);
        session.getTransaction().commit();

        Student fetched = session.get(Student.class, 1);
        System.out.println(fetched.getRollno() + " " +
fetched.getName() + " " +
        fetched.getCourse() + " " + fetched.getSemester());
        session.close();
    }
}
```

```
}
```

Output:

1 Om Rana BCA 5

Program 74: Hibernate Employee entity (empno, empnm, department) - insert & display all

Employee.java

```
@Entity
public class Employee {
    @Id private int empno;
    private String empnm, department;
    // getters and setters
}
```

MainApp.java

```
Session session = sf.openSession();
session.beginTransaction();
Employee e = new Employee();
e.setEmpno(201); e.setEmpnm("Karan Rathod"); e.setDepartment("IT");
session.save(e);
session.getTransaction().commit();

List<Employee> list = session.createQuery("from Employee",
Employee.class).list();
for (Employee emp : list)
    System.out.println(emp.getEmpno() + " " + emp.getEmpnm() + " " +
emp.getDepartment());
```

Output:

201 Karan Rathod IT

Program 75: Hibernate Product entity (id, name, price) - fetch all using HQL

Product.java

```
@Entity
public class Product {
    @Id private int id;
    private String name;
    private double price;
    // getters and setters
}
```

MainApp.java

```
Session session = sf.openSession();
Query<Product> query = session.createQuery("from Product",
Product.class);
List<Product> products = query.list();
for (Product p : products)
    System.out.println(p.getId() + " " + p.getName() + " " +
p.getPrice());
```

Output:

1 Keyboard 650.0
2 Mouse 300.0
3 Monitor 8500.0

SECTION F: Spring Framework & Spring Boot Programs (76-78)

Program 76: Simple login application using Spring MVC

LoginController.java

```
@Controller
public class LoginController {
    @GetMapping("/login")
    public String showLogin() { return "login"; }

    @PostMapping("/doLogin")
    public String doLogin(@RequestParam String uname, @RequestParam
String pass, Model model) {
        if (uname.equals("admin") && pass.equals("admin"))
            return "success";
        model.addAttribute("error", "Invalid credentials");
        return "failure";
    }
}
```

login.html (Thymeleaf template)

```
<form action="/doLogin" method="post">
    Username: <input type="text" name="uname"><br>
    Password: <input type="password" name="pass"><br>
    <input type="submit" value="Login">
</form>
```

success.html: <h3>Login Successful! Welcome Admin.</h3> failure.html:
<h3 th:text="\${error}"></h3>

Output (admin/admin):

Login Successful! Welcome Admin.

Program 77: Spring Boot + Spring Data JPA - insert & display Employee (empno,empnm,designation,dept)

Employee.java

```
@Entity
public class Employee {
    @Id private int empno;
    private String empnm, designation, dept;
    // getters and setters
}
```

EmployeeRepository.java

```
public interface EmployeeRepository extends JpaRepository<Employee,
Integer> { }
```

EmployeeController.java

```
@RestController
public class EmployeeController {
    @Autowired
    private EmployeeRepository repo;

    @PostMapping("/addEmployee")
    public Employee add(@RequestBody Employee e) { return
repo.save(e); }

    @GetMapping("/employees")
    public List<Employee> getAll() { return repo.findAll(); }
}
```

application.properties

```
spring.datasource.url=jdbc:mysql://localhost:3306/bcddb
spring.datasource.username=root
spring.datasource.password=root
spring.jpa.hibernate.ddl-auto=update
```

Output (GET /employees):

```
[
  {
    "empno": 301,
    "empnm": "Vivek Nair",
    "designation": "Developer",
    "dept": "IT"
  }
]
```

Program 78: Spring Boot + Spring Data JPA - CRUD on Student (rollno, name, course, semester)

Student.java

```
@Entity
public class Student {
    @Id private int rollno;
    private String name, course;
    private int semester;
    // getters and setters
}
```

StudentRepository.java

```
public interface StudentRepository extends JpaRepository<Student,
Integer> { }
```

StudentController.java

```
@RestController
@RequestMapping("/students")
public class StudentController {
    @Autowired
    private StudentRepository repo;

    @PostMapping
    public Student create(@RequestBody Student s) { return
repo.save(s); }

    @GetMapping
    public List<Student> getAll() { return repo.findAll(); }

    @PutMapping("/{rollno}")
    public Student update(@PathVariable int rollno, @RequestBody
```

```
Student s) {  
    s.setRollno(rollno);  
    return repo.save(s);  
}  
  
@DeleteMapping("/{rollno}")  
public String delete(@PathVariable int rollno) {  
    repo.deleteById(rollno);  
    return "Deleted rollno " + rollno;  
}  
}
```

Output (POST /students):

```
{"rollno":20,"name":"Ishaan Bhatt","course":"BCA","semester":5}
```

Output (DELETE /students/20):

Deleted rollno 20

End of CS-29 (Advance Java Programming - J2EE) Practical Solutions

Prepared as a self-study reference. Compile and run each program on a working JDK + Tomcat + MySQL setup for actual live output; the outputs shown above represent the expected/typical result of running the code with the given sample input.